Quelques exercices de rappels pour se remettre en selle

Lino Galiana
Version du 16 octobre 2026

Un chapitre consacré à divers exercices pour réviser les principes de base de la syntaxe `Python` et des objets utilisés par le langage.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Exercices de synthèse 1
1.1 Exercice 1 1
1.2 Exercice 2 3
1.3 Exercice 3: les chaînes de caractères 4
Informations additionnelles 5

1 Exercices de synthèse

Pour vérifier que vous êtes au point sur les structures de données et les opérations adéquates en `Python` pour différents problèmes, voici une série d'exercice.

Ils demandent de se creuser un peu la tête sur la structure adéquate des données pour y répondre, c'est normal !

Une fois n'est pas coutume, les exercices se font directement sur cette page plutôt que par le biais de *notebooks*[1]. La correction sera prochainement dans la page dédiée.

1.1 Exercice 1

Exercice 1

Vous disposez d'une liste de citations issues d'oeuvres littéraires de notre magnifique patrimoine littéraire :

Voir la liste de citations qui seront utiles

[1] Il est possible d'exécuter du `Python` sur le navigateur grâce à son implémentation en WASM par le biais de Pyodide. Ce n'est pas une méthode recommandée quand on doit faire des calculs conséquents comme dans les prochains chapitres. Mais comme ici nous travaillons sur de petits exemples c'est assez pratique pour bénéficier de l'interactivité permise par le navigateur, notamment les aides en cas de blocage.

Citation	Auteur
« Rien ne sert de courir ; il faut partir à point. »	La Fontaine
« Selon que vous serez puissant ou misérable, les jugements de cour vous rendront blanc ou noir. »	La Fontaine
« Heureux qui comme Ulysse a fait bon voyage »	Du Bellay
« L'homme est né libre, et partout il est dans les fers. »	Rousseau
« Parce que c'était lui, parce que c'était moi. »	Montaigne
« La première fois qu'Aurélien vit Bérénice, il la trouva franchement laide »	Aragon
« Aujourd'hui maman est morte. Ou peut-être hier, je ne sais pas. »	Camus

Créer un objet `citations` qui utilise une structure de données adéquate en `Python` pour:

1. Pouvoir facilement retrouver toutes les citations associées à un auteur. Créer un objet `citation_aragon` qui teste la validité de votre approche avec l'exemple d'Aragon.
2. Pour chaque citation, compter le nombre de mots et le nombre de mots uniques. Créer un objet `stats_phrases` qui liste ces propriétés. Tester sur les citations de la Fontaine.

Voici les citations, ici en vrac, pour pouvoir démarrer.

Dans cette cellule, créer la structure de données adéquates pour cet exercice.

i Indice 1

Quelle structure de données pourrait permettre d'avoir ce type de code ?

```
citations.get("La Fontaine")
```

i Indice 2

Quelle structure de données pourrait permettre d'avoir un objet ressemblant à

```
citations
```

i Indice 3

Un dictionnaire pourrait vous aider. Si vous avez encore du mal à comprendre pourquoi et comment, retournez au *notebook* dédié ⬆️.

```
<IPython.core.display.HTML object>
```

Pour la question 1, vous pouvez utiliser cette cellule.

Pour la question 2, vous pouvez utiliser cette cellule.

Indice 1

Comment itérer sur les clés et valeurs de votre dict `stats_phrases` ?

Indice 2

Adopter cette structure générale :

```
for auteur, liste_citations in citations.items():
    # do something
```

Comment avoir la longueur d'une phrase (suite de mots) et des mots uniques dans celle-ci ?

Indice 3

Comment avoir la longueur d'une phrase (suite de mots) et des mots uniques dans celle-ci ?

1. Une phrase (chaîne de caractères) peut être décomposée en mots (liste) avec la méthode `split()`
2. Un `set` permet de dédoublonner une liste

Indice 4

Vous devriez obtenir ce type de résultat:

```
stats_phrases
```

```
<IPython.core.display.HTML object>
```

Warning

Idéalement, il faudrait faire attention à la ponctuation.

Mais nous n'avons pas encore appris comment faire des substitutions raffinées dans des chaînes de caractères, cela sera dans un prochain chapitre.

1.2 Exercice 2

Les *advent of code* sont d'excellents problèmes pratiques pour apprendre l'algorithmique. Avec un problème par jour de difficulté croissante entre le 1er décembre et le jour de Noël, on devient très vite à l'aise avec les nombreuses structures de données qu'offre `Python`.

Le prochain exercice propose de résoudre la première partie du jour 1 (année 2022) exclusivement avec du `Python` base.

Warning

Il serait pertinent d'utiliser `Numpy` pour le prochain problème. Mais c'est l'objet du prochain chapitre de montrer la manière dont ce *package* simplifie les opérations numériques basiques.

Voici les objets dont nous aurons besoin pour cet exercice:

Exercice 2

Cet exercice vise à vous guider pour résoudre le problème sur cette page (*Advent of Code* 2022, jour 1, première partie). Il va être nécessaire de trouver une structure de données plus adéquate que celle de départ.

1. Lire la consigne
2. En allant progressivement, essayez de retrouver sur l'exemple le volume de calories portées par chaque elfe.
3. En déduire l'elfe portant le plus de calories. Vérifier que vous obtenez bien sur l'exemple les calories évoquées dans la consigne.
4. Généraliser pour résoudre le problème

Vous pouvez dans cette cellule écrire votre code pour la question 2:

Indice 1

Il va falloir itérer sur chaque valeur de `elves_example`: comment transformer la chaîne de caractère en objet plus maniable ?

Indice 2

Attention au type d'objet que vous allez obtenir, n'oubliez pas de convertir dans un format numérique si vous désirez faire des opérations arithmétiques

Indice 3

Vous devriez finir avec la liste

```
calories_example
```

```
<IPython.core.display.HTML object>
```

Vous pouvez dans cette cellule écrire votre code pour la question 3:

Pour finir, après avoir généralisé, vous devriez retrouver les valeurs suivantes:

```
Elf number 217 carries the most calories (72602 calories).
```

1.3 Exercice 3: les chaînes de caractères

Voici le type de motif final vers lequel vous désirez aboutir à la question 1 et 2.

```
On 17 March 2022, Nadia (34) reported an income of $33,333.33.
```

Exercice 3

1. A partir de la méthode `format`, essayez d'obtenir le résultat attendu.
2. Faites de même avec les *f-strings*.
3. Modifiez le message pour créer un revenu mensuel arrondi à 1 chiffre. Utiliser l'espace comme séparateur de milliers plutôt que le séparateur par défaut (la virgule).
4. Voici un chemin fictif `C:\Users\Nadia\Documents\cours\chapitre_03\notes.txt` vers un fichier. Faire un *print* correctif de celui-ci.

Dans la première partie de cet exercice, vous devriez avoir ces résultats.

```
Question 1:
On 17 March 2022, Nadia (34) reported an income of $33,333.33.
Question 2:
On 17 March 2022, Nadia (34) reported an income of $33,333.33.
Question 3:
On 17 March 2022  Nadia (34) reported an income of $2 777.8.
```

Dans la deuxième partie de l'exercice, si vous utilisez le mauvais type de *string*, vous devriez avoir une erreur:

```
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated
\UXXXXXXXX escape (3077220908.py, line 2)
  ␛[36mCell␛[39m␛[36m ␛[39m␛[32mIn[14]␛[39m␛[32m, line 2␛[39m
␛[31m    ␛[39m␛[31mchemin_bad = "C:\Users\Nadia\Documents\cours\chapitre_03\notes.txt"␛[39m
               ^
␛[31mSyntaxError␛[39m␛[31m:␛[39m (unicode error) 'unicodeescape' codec can't decode bytes in
position 2-3: truncated \UXXXXXXXX escape
```

Les slashs posent souvent problème. Avec des *raw strings* vous ne devriez pas rencontrer ce problème:

```
File can be found here: C:\Users\Nadia\Documents\cours\chapitre_03\notes.txt
```

Informations additionnelles

ℹ Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```toml
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
requires-python = "≥3.13,<3.14"
dependencies = [
    "altair≥6.0.0",
    "cartiflette≥0.2",
    "contextily==1.6.2",
    "duckdb≥0.10.1",
```

```
    "folium>=0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables>=0.12.0",
    "gt-extras>=0.0.8",
    "ipykernel>=6.29.5",
    "jupyter>=1.1.1",
    "jupyter-cache>=1.0.0",
    "kaleido>=0.2.1",
    "langchain-community>=0.3.27",
    "loguru==0.7.3",
    "markdown>=3.8",
    "nbclient>=0.10.0",
    "nbformat>=5.10.4",
    "nltk>=3.9.1",
    "openpyxl>=3.1.5",
    "pandas>=3.0",
    "pip>=25.1.1",
    "plotly>=6.1.2",
    "plotnine>=0.15",
    "polars>=1.8.2",
    "py7zr>=1.1.3",
    "pyarrow>=17.0.0",
    "pynsee>=0.1.8",
    "python-dotenv>=1.0.1",
    "python-frontmatter>=1.1.0",
    "pywaffle>=1.1.1",
    "requests>=2.32.3",
    "scikit-image>=0.24.0",
    "scikit-learn>=1.8.0",
    "scipy>=1.13.0",
    "seaborn>=0.13.2",
    "selenium<4.39.0",
    "spacy>=3.8.4",
    "webdriver-manager>=4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
```

```
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.